

Integrity Constraints in SQL

Introduction

Definition

Integrity constraints ensure that changes made to the database by authorized users do not violate data consistency and correctness. They protect the database from accidental damage and enforce business rules.

Examples of Integrity Constraints

- **Not Null:** Instructor name cannot be null.
- **Primary Key (PK):** No two instructors have the same instructor ID.
- **Foreign Key (FK):** Every department name in the course table must exist in the department table.
- **Check:** Department budget must be greater than 0.

Declaring Integrity Constraints

Declaration Methods

Integrity constraints can be declared:

- During `CREATE TABLE` statements.
- Added later using `ALTER TABLE` statements.

Integrity Constraints on a Single Relation

Common Constraints

- **NOT NULL:** Attribute cannot have null values.
- **PRIMARY KEY:** Unique identifier; not null and unique.
- **UNIQUE:** Attribute or set of attributes must be unique but can be null.
- **CHECK (predicate):** Ensures attribute values satisfy a condition.

Not Null and Unique Example

```
CREATE TABLE department (  
    name VARCHAR(20) NOT NULL,  
    budget NUMERIC(12,2) NOT NULL,  
    UNIQUE (name)  
);
```

Check Constraint

Definition

The CHECK clause specifies a predicate that must be true for each tuple.

Check Constraint Example

Ensure semester is one of 'Fall', 'Winter', 'Spring', 'Summer':

```
CREATE TABLE section (  
    course_id VARCHAR(8),  
    sec_id VARCHAR(8),  
    semester VARCHAR(6),  
    year NUMERIC(4,0),  
    building VARCHAR(15),  
    room_number VARCHAR(7),  
    time_slot_id VARCHAR(4),  
    PRIMARY KEY (course_id, sec_id, semester, year),  
    CHECK (semester IN ('Fall', 'Winter', 'Spring', 'Summer'))  
);
```

CHECK Constraint Example

Ensure semester is one of 'Fall', 'Winter', 'Spring', 'Summer':

```
CREATE TABLE section (  
    course_id VARCHAR(8),  
    sec_id VARCHAR(8),  
    semester VARCHAR(6),  
    year NUMERIC(4,0),  
    PRIMARY KEY (course_id, sec_id, semester, year),  
    CONSTRAINT chk_semester  
        CHECK (semester IN ('Fall', 'Winter', 'Spring', 'Summer'))  
);
```

NOT NULL Constraint

Definition

The NOT NULL constraint ensures that a column cannot have NULL values. Every tuple must provide a value for that attribute.

NOT NULL Example

Declare name and budget to be NOT NULL:

```
CREATE TABLE department (  
    name VARCHAR(20) NOT NULL,  
    budget NUMERIC(12,2) NOT NULL  
);
```

NOT NULL Example

Ensure that name and budget cannot be NULL:

```
CREATE TABLE department (  
    name VARCHAR(20)  
        CONSTRAINT dept_name_not_null NOT NULL,  
    budget NUMERIC(12,2)  
        CONSTRAINT dept_budget_not_null NOT NULL  
);
```

UNIQUE Constraint

Definition

The UNIQUE constraint specifies that a combination of one or more attributes must be unique across all tuples in a relation. Unlike PRIMARY KEY, UNIQUE columns can contain NULL values.

UNIQUE Constraint Example

Ensure that the combination of A1 and A2 is unique:

```
CREATE TABLE example_table (  
    A1 VARCHAR(20),  
    A2 VARCHAR(20),  
    UNIQUE (A1, A2)  
);
```

UNIQUE Constraint Example

Ensure that `name` in the `department` table is unique:

```
CREATE TABLE department (  
    name VARCHAR(20)  
        CONSTRAINT dept_name_unique UNIQUE,  
    budget NUMERIC(12,2)  
);
```

PRIMARY KEY Constraint

Definition

A **PRIMARY KEY** uniquely identifies each tuple in a relation. It implies both **UNIQUE** and **NOT NULL** constraints.

PRIMARY KEY Example

Define a composite primary key:

```
CREATE TABLE section (  
    course_id VARCHAR(8),  
    sec_id VARCHAR(8),  
    semester VARCHAR(6),  
    year NUMERIC(4,0),  
    PRIMARY KEY (course_id, sec_id, semester, year)  
);
```

PRIMARY KEY Example

Define a primary key on `course_id` and `sec_id`:

```
CREATE TABLE section (  
    course_id VARCHAR(8),  
    sec_id VARCHAR(8),  
    semester VARCHAR(6),  
    year NUMERIC(4,0),  
    CONSTRAINT pk_section PRIMARY KEY (course_id, sec_id)  
);
```

FOREIGN KEY Constraint

Definition

A **FOREIGN KEY** enforces referential integrity by ensuring that the value in one relation matches an existing value in another relation's **PRIMARY KEY** or **UNIQUE** column.

FOREIGN KEY Example

Link dept_name in course to dept_name in department:

```
CREATE TABLE course (  
    course_id CHAR(5) PRIMARY KEY,  
    title VARCHAR(20),  
    dept_name VARCHAR(20),  
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

FOREIGN KEY Example

Link dept_name in instructor table to name in department:

```
CREATE TABLE instructor (  
    ID VARCHAR(5),  
    name VARCHAR(20),  
    dept_name VARCHAR(20),  
    salary NUMERIC(8,2),  
    CONSTRAINT fk_instructor_dept  
        FOREIGN KEY (dept_name)  
        REFERENCES department(name)  
);
```

Referential Integrity

Definition

Ensures that foreign key values in one relation must exist as primary key values in another relation, maintaining consistency between related tables.

Referential Integrity Example

If "Biology" appears as a department name in `instructor`, it must appear in `department`:

```
CREATE TABLE department (  
    dept_name VARCHAR(20) PRIMARY KEY,  
    ...  
);  
  
CREATE TABLE instructor (  
    ID CHAR(10) PRIMARY KEY,  
    name VARCHAR(40),  
    dept_name VARCHAR(20),  
    FOREIGN KEY (dept_name) REFERENCES department(dept_name)  
);
```

Cascading Actions in Referential Integrity

Cascading Options

- `ON DELETE CASCADE`: Delete child rows when parent row deleted.
- `ON UPDATE CASCADE`: Update child rows when parent key updated.
- Alternatives: `SET NULL`, `SET DEFAULT`.

Cascading Foreign Key Example

```
CREATE TABLE course (  
    course_id CHAR(5) PRIMARY KEY,  
    title VARCHAR(20),  
    dept_name VARCHAR(20),  
    FOREIGN KEY (dept_name) REFERENCES department  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

Integrity Constraint Violation During Transactions

Example: Recursive Foreign Keys

```
CREATE TABLE person (  
    ID CHAR(10) PRIMARY KEY,  
    name CHAR(40),  
    mother CHAR(10),  
    father CHAR(10),  
    FOREIGN KEY (father) REFERENCES person(ID),  
    FOREIGN KEY (mother) REFERENCES person(ID)  
);
```

Inserting a tuple without violating constraints:

- Insert parents before inserting the person.
- Or insert with `mother` and `father` as NULL initially, then update after.
- Or defer constraint checking (if supported).

Complex Check Clauses

Limitations

- Subqueries inside `CHECK` clauses are generally not supported.
- Example desired: enforce `time_slot_id` must appear in `time_slot` relation.
- Ensuring "every section has at least one instructor" cannot be expressed with check constraints.
- Alternative: Use triggers or assertions (rarely supported).

Unsupported Assertion Syntax

```
CREATE ASSERTION assertion_name  
CHECK (<predicate>);
```

Note: Not supported by most SQL implementations.